

Multi-Robot Interaction

A Unified GP6–VLA Runtime over Cross-Host ROS 2

Research Proposal III · Zimo Wen

September 2026

Abstract

This proposal studies cooperation among robots whose sensors, controllers, and policy services run on different machines. GP6 produces and repairs a task directed acyclic graph (DAG); VLA policies and code skills realize its nodes; a shared runtime coordinates dependencies, observations, and control ownership; ROS 2 carries sensor streams, action goals, feedback, and cancellation across the network. The central problem is not issuing a command to one arm. It is preserving a meaningful joint task when one robot acts on older evidence, a remote action finishes late, or a communication failure makes another robot’s state uncertain. We propose resource-aware DAG execution, robot-side admission contracts, and failure-aware group coordination. The research asks whether these mechanisms improve cooperative task completion and recovery at matched policy and communication budgets. Existing local integration checks establish software feasibility only. Cross-host transport, real model inference, coupled simulation, and physical multi-robot performance require separately identified evidence.

1. Multi-Robot Interaction Is the Primary Setting

A representative task. One arm presents an object, another grasps it, and the first releases only after the second has established the required condition. Each arm has its own control computer; a separate machine can host GP6 and a shared VLA service. A network acknowledgement, an action result, and verified object transfer are different events. A system must represent the dependency between them rather than independently running two single-arm scripts.

Division of work. GP6 chooses task structure, assigns capabilities, and repairs unfinished work from permitted feedback. VLA policies produce observation-conditioned actions within declared command spaces. Code skills handle established local behavior. The runtime schedules nodes and controls ownership. Robot-side ROS nodes connect native drivers and controllers; high-frequency control remains local. None of these layers should silently invent absent sensor values or infer task success from an empty result list.

Why ROS 2 matters here. Robot identities and namespaces, sensor publication, asynchronous Actions, feedback, cancellation, and controller interfaces must work across machine boundaries. ROS 2 provides a common robot-facing communication substrate; the runtime supplies the task-level execution contract above it. Middleware discovery and message delivery alone do not provide correct handoff or joint-task recovery.

Relationship to the other proposals. Proposal I studies manipulation and force–position control. Proposal II studies generated reactive local policies. Proposal III studies their composition into a **distributed multi-robot task**. Single-arm deployment is a compatibility case. It is not the primary experiment or the justification for ROS 2.

2. Distributed Architecture and Typed Contracts

Explicit topology. A deployment names each robot, arm, gripper, sensor, control host, and supported capability. Resources are not anonymous array indices. A coordinated capability declares the union of its member resources and any shared workspace constraint. Command dimensions, joint order, units, frames, and actuator limits belong to the device adapter. The same DAG interface must support one, two, and more arms without rewriting the scheduler.

GP6 task DAG. Nodes select a capability and a code, GP6, or VLA policy. Edges express observable pre-requisites: an approach must finish before insertion, and a receiving grasp must be established before a handoff release. Independent branches may run concurrently. GP6 can inspect allowed execution summaries and propose a revised DAG; completed nodes retain their identities and records. Repair does not replay successful work under a new graph version.

Observation identity and provenance. Each observation records robot identity, source topic, acquisition time, frame, calibration version where available, and the owning execution epoch. Camera and proprioceptive freshness are checked separately. An actual zero joint position is valid numeric data; a missing sample, empty vector, or non-finite value is not replaced by zero. Cross-host freshness requires an explicit clock domain and clock-offset uncertainty, not a comparison of unrelated monotonic clocks.

Action admission. An execution request carries a unique request ID, task and node identity, capability, observation reference, resource set, command specification, and validity horizon. The coordinator checks the DAG and resource reservation; the proposed robot-side admission gate checks ownership and freshness again before dispatch. Duplicate requests refer to an existing execution record. A delayed message from a superseded owner must not become a new physical action.

VLA and GP6 share the execution boundary. VLA outputs are converted explicitly into a supported capability contract. A normalized end-effector action cannot be relabeled as an absolute joint target. Both GP6-authored node actions and VLA chunks receive measured feedback through the same path. GP6 does not participate in every servo tick; a local controller remains responsible while a remote model is thinking.

Implementation boundary. The current prototype has typed capabilities, DAG execution, local ownership epochs, coordinated dispatch, and ROS interfaces. Robot-side persistent fencing, restart reconciliation, synchronized starts, and cross-host clock validation are additional mechanisms to implement and measure. One coordinator is currently authoritative; multi-coordinator failover is not an established property.

3. Coordination and Failure Recovery

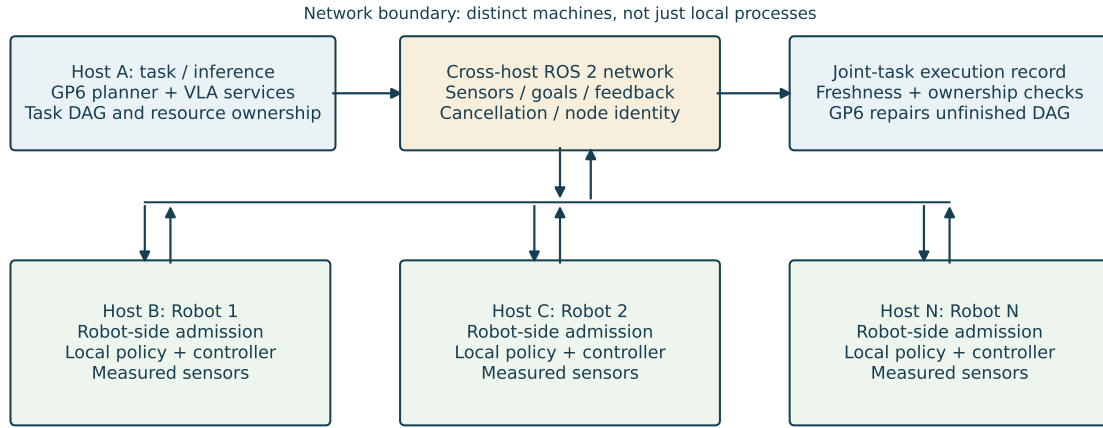


Figure 1: The network separates planning/policy services from robot control hosts. The DAG joins observations and execution results across robots. ROS 2 carries native robot messages; the runtime enforces task dependencies and ownership. A local dispatch barrier is not evidence of simultaneous physical motion.

A: dependency and resource coordination. Reserve all resources required by a node before dispatch. A multi-arm node validates every member’s arguments first; a conflict cannot leave only half the resource set reserved. Resources remain reserved through inference and execution, so another task cannot replace one arm’s reference during a pending policy call. Sensor streams can continue while execution ownership remains exclusive.

B: coupled execution and cancellation. Dispatch members concurrently and preserve their individual acceptance, progress, result, and measured-state records. If a member fails, request cancellation of the affected peers and keep the group reserved until their states are known. Cancellation being accepted does not imply that a robot has stopped. If acknowledgement is lost, quarantine the affected resources rather than treating a network timeout as a successful release. Already executed motion cannot be rolled back.

C: state-aware DAG repair. After affected controllers have settled, GP6 receives a bounded cross-robot trace and repairs the unfinished subgraph. It may retry a grasp, choose another available arm, or change a dependency. The proposal must preserve completed work, identify the relevant state change, and remain within the fixed capability and inference budgets. Global replanning and local subgraph repair are separate comparisons.

Physical timing is a separate contract. Future-stamped commands or a synchronized controller may be needed for coupled lifting or force sharing. We will measure clock uncertainty, controller admission, and actual motion onset separately. Neither Python concurrency nor ROS message timestamps prove a bounded physical start skew. A pure dispatch-barrier implementation is reported as that weaker baseline.

4. Experiments That Require More Than One Machine

Deployment ladder. First validate ROS transport on at least two separately identified hosts: a coordinator/policy host and a robot-control host. Next place two robot controllers on different hosts and run coordinated tasks. Then add a third robot or relocate the shared VLA service to test scale and placement. Record host identity, middleware configuration, route, clock domains, message provenance, and controller feedback. Several processes or containers on one host do not satisfy the cross-host experiment.

Coupled task families. Evaluate object handoff, shared-object transport, cooperative alignment, and ordered manipulation with shared workspace access. The simulation must contain a shared physical scene, so transferring or jointly supporting an object is meaningful. Independent toy plants can test transport, tracking, and cancellation, but cannot establish handoff success. Physical experiments follow only after the chosen drivers and local stop/hold behavior are established.

Planned pilot. Use six cooperative task families, three network conditions, and five initializations: 90 task episodes per method. These are proposed sample counts, not completed runs. Calibrate network disturbances and action limits on development cases; freeze held-out task–condition combinations before comparing methods. Preserve exact manifests and resume protection; successful evaluation units are not blindly rerun.

Matched baseline	Question isolated
Independent per-robot policies	Does explicit cross-robot dependency management matter?
Central GP6 DAG with code skills	What is gained by adding frozen VLA node policies?
GP6–VLA DAG over complete RPC adapters	What does ROS-based deployment add beyond a semantically matched transport baseline?
Distributed ROS DAG, local reservations	What remains unresolved without robot-side admission and coupled recovery?
Full distributed runtime	Do admission, ownership, group recovery, and selective repair improve joint outcomes?

Fair comparison. Keep task geometry, initial states, sensing, policy checkpoints, primitive library, action limits, and model budgets matched. Compare middleware only when adapter semantics are equivalent. Public scores from another harness are background references, not matched baselines. LIBERO remains a separate single-arm compatibility panel and cannot substitute for multi-robot evidence.

5. Network Stress, Metrics, and Ablations

Communication conditions. Apply controlled latency and jitter, transient observation loss, duplicate submissions, controller unavailability, and link interruption during an active coordinated action. Log actual observed delays and delivery failures. A disconnect test must show what the robot continues doing locally and when the coordinator loses confidence; it must not fabricate missing feedback or silently clear ownership.

Joint-task outcomes. Use the final independent simulator or physical harness verdict for task success. Report transfer failures, incompatible simultaneous commands, invalid/stale dispatches, intervention harm, recovery completion, and unresolved controller states. Provider, transport, image, resource, and interrupted records remain outside the task-success denominator, with their counts and system availability reported separately so exclusions do not hide deployment failures.

Timing and efficiency. Measure observation age, inference latency, action admission, motion onset where measurable, feedback arrival, cancellation request/acceptance/settlement, and recovery duration. State which timestamps share a clock. Record prompt, completion, and total tokens; VLM and unmetered calls; wall time; and perception, action, and recovery time. Compare matched task-level medians while requiring success not to regress. Lower cost caused by early failure is not an improvement.

Ablation: interaction structure. Remove cross-robot DAG edges, replace group reservation with independent per-arm submission, or disable peer cancellation in a labeled simulation diagnostic. These tests isolate coordination from the competence of a single policy. Include one-, two-, and four-arm software configurations, but do not count synthetic protocol outcomes as manipulation successes.

Ablation: policy and repair. Compare code-only, VLA-only local behavior, GP6 node actions, and their combination under the same capability interface. Compare no repair, full replanning, and unfinished-subgraph repair. Remove robot-side freshness/fencing checks in a simulation-only stress test. A gain that disappears when sensing, retries, or inference budgets are matched does not support the proposed mechanism.

Ablation: deployment placement. Hold the task and policy fixed while moving inference off the robot host and moving controllers across hosts. Compare equivalent ROS middleware configurations and a complete RPC baseline. The claim should concern measurable coordination, portability, or recovery under distribution, not that ROS intrinsically makes a language model smarter.

6. Contribution, Current Evidence, and Open-Source Scope

Intended contribution. A multi-robot execution contract that makes heterogeneous GP6, VLA, and code policies composable across machines: explicit topology, task dependencies, observation provenance, admission/fencing, coordinated ownership, and recoverable failure states. Ordinary RPC, ROS Actions, DAGs, and model tool calls are established components. Novelty must come from a validated interaction mechanism and a controlled evaluation of its consequences.

Current evidence and limits. The local Robot Runtime prototype has exercised GP6/VLA API-shaped requests, DAG joins, four controller processes, camera messages, and ROS trajectory Actions. Those initial model services and controllers were explicitly synthetic and ran on one host. They are software-integration evidence only. A separate cross-host execution journal records subsequent deployments; real model inference, physics-derived state, and final task verdicts are labeled independently. This proposal claims no new manipulation success rate.

Data integrity. Initial arrays in a test controller were explicitly initialized, not measured from hardware. Production adapters read actual ROS messages, preserve their source and timestamps, and validate shape, finiteness, units, and freshness. Execution records link to the observation that authorized them. An empty `task_verdicts` list means no task evaluator was run; it does not mean a dataset was inspected and found empty. Hidden object states and final evaluator predicates remain outside GP6, VLA, code policies, and reviewer-visible context.

Deliverables. Release the device/capability/DAG schemas, GP6 and VLA adapters, ROS gateways, versioned deployment examples for multiple hosts, protocol stress tests, and separate simulation/task-evaluation harnesses. Include 1/2/4-arm regression coverage and cross-host CI or reproducible deployment scripts. Preserve rejected requests, interrupted runs, model-call records, measured state/action traces, and code/configuration versions. Interface compatibility is a maintenance requirement, not a one-off demo property.

Interpretation. If independent policies achieve the same joint results under realistic network conditions, the coordination claim weakens. If only a better driver explains the gain, credit that engineering change. If synthetic tests pass but coupled simulation or real models fail, report the missing layer rather than upgrading the evidence label. Success on one arm does not validate this proposal’s multi-robot interaction hypothesis.

References

- [1] ROS 2 Actions: goals, feedback, results, and cancellation. <https://design.ros2.org/articles/actions.html>
- [2] ROS 2 middleware interface and deployment documentation. <https://docs.ros.org/en/jazzy/>
- [3] OpenPI: VLA policies and remote inference interfaces. <https://github.com/Physical-Intelligence/openpi>
- [4] GPT as Policy: direct and hybrid policy execution code. <https://github.com/anonymous-report-421/GPT-as-Policy>
- [5] Code as Policies: Language Model Programs for Embodied Control. <https://arxiv.org/abs/2209.07753>